Sprint 4 - Progress Report - FSS 2026

Benchmark for Structured Information Processing in Large Language Models

Arpitha Kori (2112528), Krittika Sardar (2190114)
Sushma Kulkarni (2192317), Satwik Kumar Lohani (2167219),
Nishit Suthar (2189914)

August 04, 2026

1 Hard Cases Benchmark & Framework

A curated set of Q&A pairs where LLMs consistently fail—extracted from Sprint 2–3 results rather than continuing optimisation. A fixed configuration was used: `CHUNK_SIZE=3000`, `OVERLAP=300`, `TOP_K=5`, `all-MiniLM-L6-v2` embeddings, `TEMP=0.0`, and zero-shot and CoT prompts only.

The complete `framework/` directory was built for shared use: `config.py` (fixed parameters and an eight-model registry across five API providers), `rag_runner.py` (an end-to-end RAG pipeline with provider routing and retry logic), and `scorer.py` (supporting F1, Exact Match, and Numeracy F1 for unstructured document analysis).

1.1 Benchmark Dataset Construction

All 165 failed questions extracted from Sprint 3 scored result CSVs (33 files across 6 datasets) using the criterion max F1 < 0.5 across all tested phases. Twenty music-structured questions from Sprint 2 were added, resulting in
`all_questions_combined.csv` — 185 rows across 7 datasets, with 169 flagged as hard cases. We kept ground truth side by side for all 185 rows and all PDF source paths were verified.

1.2 Multi-Model Evaluation & Dashboard

Gemini 3.1 Flash-Lite was run across all 185 questions (184/185 answered, 76 failures). The framework was extended to support OpenRouter as a provider, and Nemotron-550B and Mistral Small-2603 were run on the 76 failure questions:

Model	Answer Rate	Soft Correct	Accuracy
Gemini 3.1 Flash-Lite	98.7%	1 / 76	1.3%
Nemotron-550B (OpenRouter)	98.7%	1 / 76	1.3%
Mistral Small-2603	68.4%	1 / 76	1.3%

75 of 76 questions (98.7%) failed all three models, confirming the benchmark's robustness as a hard-case set. FinHybrid scored 0% accuracy across all models. A self-contained interactive HTML dashboard was built featuring 8 KPI cards, 6 Chart.js charts, a filterable question table with per-row answer modal, and PDF export.

2 Controlled Experimentation using IMDb Structured Data in CSV format

2.1 Benchmark Extension, Execution, and Evaluation Framework

The benchmark uses the IMDb structured subset CSV as its sole data source, excluding raw IMDb TSV files, web search, APIs, and external IMDb knowledge. Building on Sprint 3, it was expanded to 30 analytical tasks across eight categories: aggregation and statistics, ranking and Top-k, filtering and multi-step reasoning, text parsing and roles, set operations, entity grouping and collaboration, temporal reasoning, and window and contrast reasoning.

Execution Framework: For the three LLM-only modes, the complete CSV is inserted in the prompt and the model returns a CSV-formatted answer without SQL or external tools. In SQL-detour zero-shot mode, the same CSV is loaded into an in-memory SQLite table, where the model generates a validated read-only query whose result becomes the final response.

The runner script processed 30 tasks across three models (**NVIDIA Nemotron-3-Ultra-550B-A55B, Gemini 3.1 Flash Lite, Mistral Small 4 119B 2603**) and four prompt modes, producing 360 combinations. Temperature was fixed at 0, provider-specific reasoning was disabled, and the setup used a 600-second timeout, three attempts, a 10-second retry interval, and a 30,000-token output limit. Completion guards and repeated-output detection identified truncated, malformed, or degenerate responses. Ground truths were withheld from the models and used only for evaluation.

Evaluation Framework: The evaluation script loads the four execution-history files for each model and compares every available response with its corresponding PostgreSQL-generated ground-truth CSV. It separates successful execution from answer correctness and calculates content exact match, strict task success, row-level precision, recall and F1 score. It also checks output-format validity and forbidden-method violations and records execution time and token usage. The resulting task-level, mode-level and cross-mode metrics are stored in a hierarchical JSON file and a model-specific Excel master table.

2.2 Evaluation Results across Models and Modes

Average F1 scores and exact matches across models and prompt modes

	LLM-only F1 (%)			SQL-d	Exact Matches
Model	**Zero-shot**	**One-shot**	**Few-shot**	**F1 (%)**	**(SQL-d vs GT)**
Nemotron	7.50	6.96	6.60	41.77	5/30
Gemini	5.63	4.61	5.49	34.71	6/30
Mistral	0.77	0.06	1.05	39.50	4/30

In the above table, LLM-only, SQL-d and GT signifies LLM-only mode, SQL-detour mode and Ground-Truth respectively. None of the models achieved an exact ground-truth match in any LLM-only mode, and adding examples did not consistently improve performance. SQL-detour produced substantially higher F1 scores; however, the reported average F1 scores were calculated only over successfully evaluated tasks: 24 for Nemotron, 28 for Gemini, and 14 for Mistral. For detailed analysis, refer to the dashboard.

2.3 Task Category Analysis

Across all three LLM-only modes, the models struggled in nearly every category and produced no exact matches. Aggregation and statistics was the main weakness, while set operations, temporal reasoning, entity grouping, and window-and-contrast tasks yielded little or no correct row-level output. Ranking and filtering showed limited partial success, but one-shot and few-shot prompting did not consistently outperform zero-shot prompting.

SQL-detour improved performance but remained weak on complex tasks. Aggregation achieved only 15.13% F1 for Nemotron and Gemini and 2.63% for Mistral, while set operations reached 36.22%, 36.22%, and 0%, respectively. Filtering scored 33.33% across all models, largely because T08 was solved correctly. Mistral particularly struggled with entity and temporal reasoning and had 16 SQL execution failures. Window-and-contrast reasoning could not be assessed reliably because of failed executions and missing ground-truth links.

Overall, SQL-detour improved computation over direct CSV reasoning, but grouped aggregation, complex joins, set logic, and multi-stage conditions still caused incorrect queries, incomplete results, and execution failures.

3 Hotpot QA: Structured Knowledge Retrieval vs. Direct Inference

3.1 Dataset

We evaluate on the HotpotQA distractor validation split (7,405 examples), where each example pairs a question with ten Wikipedia documents — two gold supporting passages

and eight distractors. The distractor split guarantees gold documents are always present, ensuring failures reflect reasoning quality rather than retrieval gaps. Rather than using the original two-hop questions, we generated 93 original harder questions by providing the LLM all ten documents and instructing it to invent a question requiring cross-document synthesis. The benchmark spans ten reasoning categories including Temporal Reasoning, Corporate Chain Reasoning, and Evidence Synthesis.

3.2 Approaches

Approach A (Direct LLM) concatenates all ten documents with the question into a single prompt and issues one API call, requesting a structured JSON answer. **Approach B (Structured SQL)** operates in two stages: an offline extraction stage that converts each document into subject-relation-object triples stored in a SQLite knowledge base, and an online inference stage that generates a `SELECT` query from the question — augmented with 20 sampled stored facts to bridge the vocabulary gap between extraction and retrieval — before synthesising a final answer from the returned rows. All experiments use NVIDIA Nemotron-3-Ultra-550B-A55B at temperature 0.0 with thinking mode disabled.

3.3 Results

Both approaches achieve 45% exact match (EM) overall (Approach A: F1 = 0.497; Approach B: F1 = 0.479), but the per-category breakdown reveals clear complementarity. The structured pipeline outperforms the direct baseline on retrieval-heavy tasks — Evidence Synthesis (100% vs. 67%), Corporate Chain Reasoning (67% vs. 33%), and Spatial Reasoning (50% vs. 33%) — where answers are discrete entities reachable by triple traversal. The direct LLM dominates on Temporal and Numeric Reasoning (both 100% vs. $\leq$50%), where answers are verbatim values requiring no cross-document chaining. Failures in the structured pipeline fall into three categories: over-constrained `AND`-chained SQL returning zero rows, overly broad `OR` queries returning noisy result sets, and answers absent from the triple store due to incomplete extraction. These are engineering limitations with concrete fixes — OR-with-reranking, result-set capping, and hybrid dense retrieval — rather than fundamental architectural flaws.

4 IMDb Evidence-Structure Benchmark

A controlled IMDb benchmark was developed to study how Large Language Models perform when the same information is presented in different evidence formats. The experiment started with the most difficult format, a **screenshot-based PDF**, and then moved progressively toward more structured evidence: **HTML**, **Clean CSV**, and **SQL**. This pipeline was designed to separate extraction difficulty from reasoning difficulty.

The benchmark used **20 IMDb movie pages** containing metadata such as title, year, runtime, genre, director, IMDb rating, vote count, and top cast. The initial screenshot-PDF experiment used **10 broad questions across 10 categories** to identify where the models failed. Mistral Small 4 achieved **1/10 correct answers**, while Gemini 3.5

Flash achieved **5/10 correct answers**. This confirmed that screenshot evidence creates both visual extraction difficulty and analytical reasoning difficulty.

4.1 Refined Failure-Focused Benchmark

Based on the screenshot-PDF failures, the benchmark was refined into **14 questions across 11 category labels**. Instead of testing every possible category equally, the refined benchmark focused on the categories where the models showed repeated failures. The refined questions tested arithmetic aggregation, statistical outlier detection, grouped aggregation, multi-condition filtering, genre-level aggregation, and cast-based reasoning.

The refined benchmark was then tested on semi-structured **HTML evidence** and clean **CSV evidence**. HTML reduced visual extraction difficulty because IMDb metadata was available in webpage structure and JSON-LD blocks. CSV further simplified the evidence by presenting the same information as clean structured rows and columns.

Evidence Phase	Model / Status	Correct	Partial	Incorrect	Strict Accuracy
Screenshot PDF	Mistral Small 4	1/10	0/10	9/10	10.00%
Screenshot PDF	Gemini 3.5 Flash	5/10	1/10	4/10	50.00%
HTML refined	Mistral Small 4	3/14	6/14	5/14	21.43%
HTML refined	Gemini 3.5 Flash	10/14	1/14	3/14	71.43%
Clean CSV	Mistral Small 4	6/14	3/14	5/14	42.86%

The results show that cleaner evidence improved performance, especially for Gemini on HTML and Mistral on CSV. However, Mistral still made reasoning errors even with clean structured CSV, indicating that some failures were not only caused by extraction difficulty but also by analytical reasoning limitations.

4.2 SQL Ground Truth and Final Failure Categories

A SQLite database was created as the executable ground-truth layer. SQL was used to validate the clean CSV answers through deterministic queries. The SQL-vs-CSV comparison showed **13 exact matches**, **1 format-specific wording difference**, and **0 real mismatches** across 14 refined questions. This confirmed that the structured CSV ground truth was reliable.

Based on strict accuracy across the completed refined runs, three main failure categories were selected for the next benchmark phase:

These categories are not IMDb-specific. They represent common analytical tasks: exact numerical calculation, rule-based outlier detection, and grouped aggregation. The next step is to rerun the pending Gemini CSV experiment once the API becomes available and then test Phase 2 approaches such as structured detour prompting, loop-based self-checking, and SQL-assisted verification.

Final Category	Matched / Total	Strict Accuracy	Reason for Selection
Arithmetic aggregation	0/6	0.00%	Exact calculations failed repeatedly.
Statistical outlier detection	3/6	50.00%	Requires multi-step numerical rule application.
Genre-level aggregation	2/6	33.33%	Requires grouping, filtering groups, averaging, and ranking.

5 Looping Evaluation Pipeline for Structured Information Retrieval

This sprint introduced a **looping evaluation pipeline** — a bounded retry framework that allows an LLM up to three attempts per question, injecting structured feedback after each wrong answer. The goal was to measure whether feedback-driven self-correction improves structured information retrieval, and which type of feedback is most effective.

5.1 Pipeline Design

The pipeline was built as a 9-module system: `run.py` (entry point with CLI arguments), `evaluator.py` (model API abstraction supporting OpenAI-compatible and Gemini backends), `loop_controller.py` (core retry engine), `feedback.py` (feedback message generator), `classifier.py` (rules-based failure mode classifier), `reporter.py` (crash-safe CSV saving and metric computation), `documents.py` (context builder), `config.py` (central configuration), and `compare.py` (cross-run analysis). Key design features include async execution with concurrent workers, a canonical normaliser that strips $, commas, and % before exact-match comparison, and crash-safe row-by-row saving that enables run resumption.

Three feedback strategies were implemented: **Blind** — a generic retry prompt with no directional hint; **Guided** — directs the model to a specific section of the document based on the question category; and **Structured** — provides an explicit step-by-step decomposition tailored to the question type. An **Auto** routing mode was also implemented, which selects the optimal strategy per category using a pre-defined routing table.

5.2 Benchmark Datasets

Two datasets were used. The **Music benchmark** comprises 30 questions across 6 categories (Aggregation, Ranking/Top-k, Genre-level Summary, Conditional Filtering, Multi-hop/Subquery, and Grouping/Multi-step Reasoning), grounded in 10 synthetic artist prose profiles with engineered anomalies. The **F1 Racing benchmark** comprises 55 questions across 11 categories, grounded in 17 trimmed race documents. Gold-standard answers for both benchmarks were verified using a Neon PostgreSQL database.

5.3 Evaluation and Results

NVIDIA Nemotron Ultra 550B was fully evaluated on both datasets across all three strategies (6 complete runs, 255 total questions answered). Gemini 3.5 Flash-Lite was evaluated on both datasets with the Guided strategy. Table ?? summarises the results.

Model	Dataset	Strategy	Qs	Pass-1	Final Acc	Self-Corr	DNF
Nemotron Ultra	Music	Blind	30/30	50.0%	50.0%	1	15
Nemotron Ultra	Music	Guided ⋆	30/30	46.7%	**63.3%**	5	11
Nemotron Ultra	Music	Structured	30/30	46.7%	50.0%	1	15
Nemotron Ultra	F1	Blind	55/55	58.2%	63.6%	3	20
Nemotron Ultra	F1	Guided ⋆	55/55	58.2%	**69.1%**	6	17
Nemotron Ultra	F1	Structured	55/55	60.0%	65.5%	3	19
Gemini Flash-Lite	Music	Guided	28/30	78.6%	**89.3%**	3	1
Gemini Flash-Lite	F1	Guided	39/55	74.4%	74.4%	0	10

Looping pipeline results. ⋆ denotes best strategy per dataset for Nemotron Ultra.